AGENT-READY PRODUCT SPECIFICATION

Intelligent Approval Automation System

Approval Automation Agent

Version	1.0
Build Plan	3 Phases over 16 weeks
Stack	FastAPI / Supabase / Python / n8n
Auth	Auth
Last Updated	March 2026

Read this entire specification before writing a single line of code.

This document is the single source of truth for all AI coding agents. When the spec and your judgment conflict, implement the spec and add a // SPEC QUESTION: comment.

Table of Contents

1. Product Overview
2. Tech Stack
3. Project File Structure
4. Data Models
5. Phase 1 — MVP: Core Timer & Session Log (Weeks 1–4)
6. Phase 2 — Growth: Projects, Analytics & Goals (Weeks 5–10)
7. Phase 3 — Intelligence: AI Insights & Coach (Weeks 11–16)
8. API Reference (Full)
9. UI/UX Guidelines
10. Error Handling Standards
11. Non-Functional Requirements
12. Environment Variables
13. Glossary
14. Agent Instructions & Constraints

Product Overview

Intelligent approval automation system is an automatic approval system powered by AI. The system serves as an intermediate between a host and a user. Users will submit a request for something which the business provides (ex. a user asks for a reservation at a room). An AI agent will determine the “risk” of the request. Low-risk requests will be automatically approved, while high-risk requests will be elevated to the business’ representative for manual review.

The purpose of this product is to speed up the process of basic interactions. What would normally be a long back-and-forth between business and user for trying to book a room, schedule something, or otherwise do something which requires approval, can now be swiftly taken care of with the product. If something requires more details, a direct conversation could be started.

1.1 Target Users

- Businesses or organizations which regularly have scheduled meetings, reservations, or other things which require approval
- Small groups wanting a simple organization system
- Users looking to send requests to businesses/organizations

1.2 Core Value Proposition

- Simplicity: users should not have trouble submitting requests
- Speed: users should know whether their requests are accepted within a minute*
- Secure, logical AI analysis which differentiates between “high” and “low” risk requests

Platform	Web application
Auth	Supabase Auth
Persistence	Server-side storage of requests/messages; client-side sync
Offline	Requests/messages should be stored offline (in case of emergency)
Environments	Development (local) - Staging - Production

2. Tech Stack

All technology decisions below are fixed. Do not substitute libraries without adding a // ARCH DECISION: comment explaining the reason.

Layer	Technology	Notes / Constraints
Frontend	React	
Styling	Tailwind CSS	
Backend	Python	
Database	Supabase	
Auth	Supabase Auth	
Workflow Mgmt.	n8n	Connects with AI agent
AI (Phase 3)	OpenAI API	
Testing	Pytest	
Hosting		

3. Project File Structure

Create files in this exact structure. Do not reorganize without a // ARCH DECISION: comment.

```
email-agent-reader-Backend/
├── backend/
│   └── app/
│       ├── main.py             # FastAPI application and route
definitions
│       ├── supabase_client.py  # Supabase client initialization
│       └── static/             # Colorful frontend (HTML/CSS/JS)
├── Documents/
│   ├── Links.txt               # Project-related links
│   └── Weekly Reports/         # Team weekly progress reports
└── README.md
```

4. Data Models

All models defined in Prisma schema. The agent must implement these exactly. Fields marked Phase 2 or Phase 3 should be added via migration at the start of that phase — do not add them in Phase 1.

4.1 User

```
model User {
  id              String        @id @default(cuid())
  email           String        @unique
  name            String
  googleId        String?       @unique  // Phase 1: OAuth

  createdAt       DateTime      @default(now())
  updatedAt       DateTime      @updatedAt
  settings        UserSettings?
  emailRequests   emailRequests[]

}
```

4.2 EmailRequests

```
model emailRequests {
  id                  String   @id @default(cuid())
  senderEmail         String   @unique
  subject             String
  Body                String

  Status              String @default("pending")
  recievedAt          DateTime @default(now())

  userID              String

  User User           @relation(fields: [userId], references: [id])
  approvalDecision ApprovalDecision?
  aiAnalysis AIAnalysis? // Phase 3
}
```

4.3 ApprovalDecision

```
model ApprovalDecision {
  id           String       @id @default(cuid())
  decision     String
  createdAt    DateTime?    // null = session still active

  emailRequest String
  emailRequest EmailRequest @relation(fields: [emailRequestId], references:
[id])

}
```

4.4 AIAnalysis (Phase 3)

```
model AIAnalysis {
  id             String    @id @default(cuid())


  category String
  riskLevel String
  confidenceScore Float

  summary String
}
```

PHASE 1

1. Core Email Processing & Approval Workflow

Weeks 1–4 · Goal: Build a working system that receives email requests, processes them through the workflow, and records approval decisions.

Phase 1 is complete when: The system successfully receives an email request through the backend, processes it through the n8n workflow, stores the request and decision in Supabase, and displays the processed request in the system interface.

F-101: Email Intake

ID	Requirement	Acceptance Criteria
R-101	System receives email request	Email request is received through API endpoint or webhook..
R-102	Email request data is stored	Email sender, subject, and body are stored in the database.
R-103	System creates a request record	A new EmailRequest record is created in Supabase when a request arrives.

F-102: Workflow Processing

ID	Requirement	Acceptance Criteria
R-201	Email request triggers workflow	Incoming request triggers the n8n workflow automatically.
R-202	System evaluates request risk	Workflow assigns a risk level to the request (Low, Medium, High).
R-203	System generates approval decision	Workflow produces either an Approve or Reject decision.

F-103: Request Logging

ID	Requirement	Acceptance Criteria
R-301	Approval decision is stored	ApprovalDecision record is stored in database.
R-302	Email request status is updated	Request status updates to approved or rejected.
R-303	Requests can be viewed in system	System can display stored email requests and decisions.

Phase 1 — Out of Scope

- AI-generated email summaries
- Advanced risk analysis
- Manual approval overrides
- Email notifications
- User analytics dashboard

Phase 1 — Build Order

1. Database schema (User, EmailRequest, ApprovalDecision models)
2. Backend API for receiving email requests
3. n8n workflow setup
4. Integration between backend and n8n
5. Store workflow results in Supabase
6. Basic interface for viewing processed requests
7. Basic system testing

PHASE 2

2. Growth — Projects, Analytics & Goals

Weeks 5–10 · Goal: Expand the system to allow better monitoring, tracking, and manual management of approval requests.

Prerequisite: All Phase 1 requirements must pass before starting Phase 2. Existing email request data must be preserved and database migrations should not overwrite stored records.

Phase 2 is complete when: Users can view stored approval requests, filter requests by status or risk level, and manually review or override automated approval decisions.

Phase 2 — Database Migrations Required

- Add `riskLevel` field to EmailRequest
- Add `decisionReason` field to ApprovalDecision
- Create request history logging
- Add timestamps for workflow processing events

F-201: Request Management

ID	Requirement	Acceptance Criteria
R-401	User can view stored email requests	System displays stored email requests with status and risk level.
R-402	User can filter requests	Requests can be filtered by status (pending, approved, rejected).
R-403	User can view request details	Clicking a request displays sender, subject, body, and decision.

F-202: Manual Decision Override

ID	Requirement	Acceptance Criteria
R-501	User can override automated decision	User can manually change approval decision.

R-502	Override is logged	System records when and why a decision was overridden.
R-503	Updated decision is stored	Database updates ApprovalDecision record accordingly.

F-203: Workflow Monitoring

ID	Requirement	Acceptance Criteria
R-601	System logs workflow events	Workflow processing events are recorded.
R-602	Users can view workflow history	Request history shows timestamps for processing steps.

F-204: Post-Session Reflection (mood rating)

ID	Requirement	Acceptance Criteria
R-701	Post-session modal includes mood rating	1–5 star rating added to the existing post-session modal (see R-302). Stars rendered as interactive icons. Optional — user can skip without selecting.
ID	**Requirement**	**Acceptance Criteria**
R-702	Mood data stored and visible	Star rating visible in session list rows. Average mood shown in analytics summary card.

Phase 2 — Build Order

10. Run Phase 2 migrations (mood field, Project table, DailySummary table)
11. Project API (GET, POST /projects, PATCH, DELETE /projects/:id)
12. Project UI (ProjectPicker on timer, ProjectsPage in settings)
13. Analytics API (GET /analytics/summary, /by-day, /by-hour, /by-project)
14. Analytics UI (StatsCards, BarChart, DonutChart, HeatmapChart)
15. Daily goal ring on dashboard
16. Streak calculation logic + display
17. DailySummary cron job
18. Mood rating in PostSessionModal
19. Tests for all new routes and logic

PHASE 3

3. Intelligence — AI Email Analysis & Decision Assistance

Weeks 11–16 · Goal: Leverage accumulated data for personalized AI coaching.

Prerequisite: All Phase 2 requirements must pass. User must have at least 2 weeks of session data for AI features to be meaningful. AI features degrade gracefully with less data.

Security: The Anthropic API key must NEVER appear in client-side code, browser network requests, or server logs. Store in ANTHROPIC_API_KEY env var. Call Claude API only from the backend.

Phase 3 is complete when: weekly summary generates and displays; NL session entry parses at least 3 of 5 test inputs correctly; coach chat sends and receives with user context; no API key exposed client-side.

Phase 3 — Database Migrations Required

- Create **AIAnalysis** table
- Add `aiCategory` field to EmailReques
- Add `confidenceScore` field to AIAnalysis

F-301: AI Email Analysis

ID	Requirement	Acceptance Criteria
R-801	AI analyzes email content	AI receives email subject and body for analysis.
R-802	AI generates request category	AI categorizes request type (ex: booking, financial request, administrative request).
R-803	AI estimates risk level	AI assigns risk level such as Low, Medium, or High.
R-804	AI generates request summary	AI produces a short summary describing the request.

F-302: AI Decision Assistance

ID	Requirement	Acceptance Criteria
R-901	AI provides decision recommendation	AI suggests approval or rejection based on analysis.
R-902	Recommendation stored in database	AI recommendation is saved with the request record.
R-903	Users can view AI insights	System displays AI summary, category, and risk level for each request.

Phase 3 — Build Order

20. Create AIAnalysis database model
21. Integrate AI service with backend or workflow automation system
22. Implement AI email analysis within workflow
23. Store AI analysis results in database
24. Display AI insights in system interface
25. Perform system testing and validation

8. API Reference (Full)

Base URL: /api/v1. All responses are JSON. All protected routes require Authorization: Bearer <accessToken>. Timestamps are ISO 8601 UTC.

8.1 Auth Routes

POST /auth/register

Request body	{ email: string, password: string, name: string }
Success 201	{ user: { id, email, name }, accessToken: string }
Sets cookie	refreshToken (httpOnly, secure, sameSite=strict, maxAge=7d)
Error 409	Email already registered
Error 422	Validation failure — field-level errors in details

POST /auth/login

Request body	{ email: string, password: string }
Success 200	{ user: { id, email, name }, accessToken: string }
Sets cookie	refreshToken (httpOnly, secure, sameSite=strict, maxAge=7d)
Error 401	Invalid email or password. Generic message only — never reveal if email exists.
Rate limit	Max 10 requests per minute per IP. Returns 429 when exceeded.

GET /auth/google + GET /auth/google/callback

Flow	Standard Passport.js Google OAuth 2.0 flow
On success	Upsert User (match on googleId or email). Set refreshToken cookie. Redirect to /app/timer.
On failure	Redirect to /login?error=oauth_failed

POST /auth/refresh

Request	No body. Reads refreshToken cookie.
Success 200	{ accessToken: string }
Error 401	Missing, expired, or invalidated refresh token. Client must re-login.

POST /auth/logout

Auth	Protected
Success 200	{ message: "Logged out" }
Side effects	Clears refreshToken cookie. Removes refresh token hash from DB.

8.2 Session Routes (all protected)

POST /sessions — Start session

Body	{ taskName: string, plannedMins: number, sessionType: SessionType, projectId?: string }
Success 201	{ session: FocusSession }
Constraint	taskName: 1–100 chars. plannedMins: 5–180. sessionType must be a valid enum value.
Constraint	Only one active session per user (endedAt is null). Returns 409 CONFLICT if one already exists.

PATCH /sessions/:id/complete

Body	{ actualMins: number, notes?: string, mood?: number }
Success 200	{ session: FocusSession }
Constraint	notes: max 500 chars, HTML stripped. mood: integer 1–5 only.
Constraint	Cannot complete an already-ended session — returns 409.
Side effects	Sets endedAt = now(), completed = true

PATCH /sessions/:id/abandon

Body	None
Success 200	{ session: FocusSession }
Side effects	Sets endedAt = now(), completed = false, actualMins = Math.floor((now - startedAt) / 60000)
Constraint	Cannot abandon an already-ended session — returns 409.

GET /sessions — List sessions

Query params	from: ISO date, to: ISO date, projectId?: string, page: number (default 1), limit: number (default 20, max 100)
Success 200	{ sessions: FocusSession[], total: number, page: number, totalPages: number }
Default range	Last 30 days if from/to omitted

DELETE /sessions/:id

Auth	Protected. Must own the session.
Success 200	{ message: "Session deleted" }
Error 403	Session belongs to a different user
Note	Hard delete. Cannot be undone.

8.3 Project Routes (all protected)

GET /projects

Query params	includeArchived: boolean (default false)
Success 200	{ projects: Project[] }

POST /projects

Body	{ name: string, color?: string }
Success 201	{ project: Project }
Constraint	name: 1–50 chars. color: valid #RRGGBB hex string. Max 50 projects per user (returns 409 if exceeded).

PATCH /projects/:id

Body	{ name?: string, color?: string, archived?: boolean }
Success 200	{ project: Project }

DELETE /projects/:id

Success 200	{ message: "Project deleted" }
Side effect	Sets projectId = null on all linked FocusSessions. Does NOT delete sessions.

8.4 Analytics Routes (all protected)

GET /analytics/summary

Query params	from: ISO date, to: ISO date
Success 200	{ totalFocusMinutes, totalSessions, completionRate, averageDailyMinutes, currentStreak, longestStreak, averageMood }
Definitions	completionRate = completed FOCUS sessions / total FOCUS sessions. currentStreak = consecutive calendar days with >= 1 completed FOCUS session through today. averageMood = average of all non-null mood values, null if none.

GET /analytics/by-day

Query params	from: ISO date, to: ISO date
Success 200	{ data: [{ date: string (YYYY-MM-DD), focusMinutes: number, sessions: number }] }
Note	Returns one entry per calendar day in range, even if 0 minutes.

GET /analytics/by-hour

Query params	from: ISO date, to: ISO date
Success 200	{ data: [{ hour: number (0–23), focusMinutes: number, sessions: number }] }
Note	Returns entries for all 24 hours, even if 0 minutes.

GET /analytics/by-project

Query params	from: ISO date, to: ISO date
Success 200	{ data: [{ projectId: string | null, projectName: string | null, color: string | null, focusMinutes, sessions }] }
Note	Sessions with no project: projectId = null, projectName = 'No Project', color = '#9CA3AF'.

GET /analytics/heatmap

Query params	None (always last 90 days)
Success 200	{ data: [{ dayOfWeek: number (0=Mon), hour: number (0–23), focusMinutes: number }] }
Note	Returns 168 entries (7 days × 24 hours). All entries included even if 0 minutes.

8.5 Settings Routes (all protected)

GET /settings

Success 200	{ settings: UserSettings }
Note	If no settings row exists, return defaults in-memory without writing to DB.

PUT /settings

Body	Partial<UserSettings> (any combination of fields)
Success 200	{ settings: UserSettings }
Constraints	focusDuration: 1–120. shortBreakDuration: 1–30. longBreakDuration: 1–60. sessionsUntilLong: 1–10. dailyGoalMinutes: 30–480.

8.6 AI Routes — Phase 3 (all protected)

GET /ai/weekly-summary

Success 200	{ summary: string, generatedAt: string, isNew: boolean }
Logic	If AIInsight exists for current weekStartDate, return it. Otherwise generate, store, and return.
Error 204	Return 204 No Content if user has < 3 FOCUS sessions in the current week.

POST /ai/weekly-summary/regenerate

Success 200	{ summary: string, generatedAt: string }
Constraint	Max 1 regeneration per user per calendar day. Returns 429 if already regenerated today.

POST /ai/parse-session

Body	{ text: string }
Success 200	{ taskName: string, plannedMins: number, projectId: string | null, confidence: "high" | "low" }
On failure	Returns 200 with confidence: 'low' and null fields. Never return an error for parsing failures.

GET /ai/coach/messages

Success 200	{ messages: ChatMessage[] }
Note	Returns last 50 messages, oldest first (for display order).

POST /ai/coach/messages

Body	{ content: string }
Success 200	{ userMessage: ChatMessage, assistantMessage: ChatMessage }
Context	Backend assembles: system prompt with user data (last 90 days, max 3000 tokens), + last 20 messages from DB, + new user message.
Constraint	content: 1–2000 chars. Rate limit: max 30 coach messages per user per day.

9. UI/UX Guidelines

All UI built with Tailwind CSS and shadcn/ui. No custom CSS files. No inline styles except where Tailwind cannot achieve the result.

9.1 Layout

- Authenticated app: fixed sidebar (240px) on left, main content fills remaining width
- Sidebar: FlowState logo, nav links (Timer, Sessions, Analytics, Coach), user avatar + sign out at bottom
- Mobile (<768px): sidebar collapses to a bottom tab bar with icon + label for each main nav item
- Max content width: 1200px, centered with auto margins
- All data-fetching states must show skeleton loaders — no blank screens or spinners alone

9.2 -

- -

9.3 Color System

Token	Hex	Usage
Background 1	#0f1021	
Background 2	#191a3a	
Accent 1	#ff5f6d	
Accent 2	#ffc371	
Accent 3	#5d5fef	
Text	#f6f7ff	
Muted	#bec3ff	

9.4 Component Rules

- Use shadcn Dialog for ALL confirmation prompts. Never browser confirm().
- Use shadcn Toast for all transient feedback (saved, deleted, error, copied, etc.).
- All destructive actions (delete session, delete project, abandon, account deletion) require a Dialog confirmation.

- Form validation errors appear inline below the relevant field — never as page-level alerts.
- Color is never the sole means of conveying state — always pair with a text label or icon.
- All images have alt text. All interactive elements are keyboard navigable.

10. Error Handling Standards

10.1 API Error Response Format

Every API error must return this exact JSON shape. No exceptions.

```
{
  "error": {
    "code": string,      // machine-readable, e.g. 'VALIDATION_ERROR'
    "message": string,   // human-readable, safe to show to users
    "details"?: any      // optional: field-level errors for form validation
  }
}
```

10.2 Standard Error Codes

VALIDATION_ERROR	Input failed Zod validation. Include details with per-field messages.
UNAUTHORIZED	Missing, invalid, or expired access token.
FORBIDDEN	Token valid but authenticated user does not own this resource.
NOT_FOUND	Resource does not exist or belongs to another user.
CONFLICT	e.g. Email already registered. Session already active. Regeneration limit hit.
RATE_LIMITED	Too many requests. Include Retry-After header.
INTERNAL_ERROR	Unexpected server error. Log the full error server-side. Never expose stack traces.

10.3 Frontend Error Handling

- Network / 500 errors: show Toast: 'Something went wrong. Please try again.'
- 401 responses: auto-attempt token refresh once. If refresh fails, clear state and redirect to /login.
- 403 responses: show a dedicated '403 Forbidden' page.
- 404 responses: show a '404 Not Found' page with a link back home.
- Form errors (VALIDATION_ERROR): render details field-by-field inline.
- Never expose raw error objects, stack traces, or internal codes in the UI.
- Log all errors to the browser console in development. Silent in production.

11. Non-Functional Requirements

11.1 Security

- Passwords hashed with bcrypt, cost factor 12. Never store in plaintext.
- Access tokens expire in 15 minutes. Refresh tokens expire in 7 days.
- Refresh token hash stored in DB to allow server-side invalidation.
- All API routes verify the authenticated userId matches the resource owner before any read or write. No IDOR vulnerabilities.
- Rate limit login endpoint: max 10 requests per minute per IP.
- Rate limit coach messages: max 30 per user per day.
- Sanitize all user-provided strings. Strip HTML from notes and task names before storing.
- CORS: only allow requests from FRONTEND_URL in production.
- Never log passwords, tokens, cookies, or the ANTHROPIC_API_KEY.
- Anthropic API key used only in backend. Never referenced in frontend code.

11.2 Performance

- Timer drift: must not exceed 1 second per 25-minute session.
- Dashboard LCP < 2.5 seconds on a standard connection.
- API response time < 300ms for all non-AI endpoints under normal load.
- DB indexes required on: FocusSession(userId, startedAt), FocusSession(userId, projectId), DailySummary(userId, date).
- Frontend initial bundle < 500KB gzipped.
- Analytics queries must never do full table scans — always filter on indexed userId first.

11.3 Accessibility

- All interactive elements keyboard-navigable with visible focus rings.
- Timer countdown announced via aria-live='polite' region (updates every minute, not every second to avoid noise).
- WCAG 2.1 AA minimum text contrast ratios throughout.
- Color is never the sole means of conveying information.
- All form inputs have associated labels.

11.4 Code Quality

- TypeScript strict mode. No implicit any anywhere.
- All API handler functions have explicit return types.
- No console.log in production code — use the logger utility in backend/src/utils/logger.ts.
- All environment variables accessed through the Zod-validated config object in backend/src/utils/config.ts.
- Minimum test coverage: 70% for utility functions and API routes.
- Three environments: development (local), staging (Vercel preview with separate DB), production.

11.5 User Flows

New User Onboarding

29. User lands on / (root). Marketing landing page with product headline and 'Get Started' CTA.
30. Clicks 'Get Started' → /register. Fills in name, email, password (or Google button).
31. On success: redirect to /app/timer. One-time tooltip: 'Name your task and hit Start to begin your first focus session.'
32. UserSettings row created with defaults on first login.

Core Timer Loop

33. User on /app/timer. Timer shows IDLE at configured duration.
34. User optionally selects project. Enters task name. Clicks 'Start'. POST /sessions called.
35. Timer counts down. User may pause/resume. Tab title reflects countdown.
36. Timer hits 0:00. Chime plays. PostSessionModal appears.
37. User adds optional notes + mood rating. Clicks 'Save & Continue' or 'Skip'.
38. PATCH /sessions/:id/complete called. Modal closes. Timer resets to next session type.
39. If autoStartBreaks = true, break timer starts automatically.

Session Deletion

40. User on /app/sessions. Clicks Delete on a row.
41. Dialog: 'Delete this session? This cannot be undone.' Confirm / Cancel.
42. On confirm: DELETE /sessions/:id called. Row removed from list with animation.

12.

Environment Variables

Create .env.example with all keys and no values. Config must be validated via Zod at backend startup — fail fast if any required variable is missing.

```
# ── Backend (.env) ──────────────────────────────
DATABASE_URL=
JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_CALLBACK_URL=
PORT=
FRONTEND_URL=
NODE_ENV=                  # development | staging | production

# Phase 3 only:
ANTHROPIC_API_KEY=

# ── Frontend (.env) ─────────────────────────────
VITE_API_BASE_URL=
VITE_GOOGLE_CLIENT_ID=     # For OAuth button display only
```

13.

Glossary

These terms have precise meanings in this codebase. Use them consistently in code, variable names, and comments.

Term	Definition
Focus Session / FOCUS	A timed work interval. The primary unit of the app. NOT a break.
Session (generic)	Any FocusSession record, including FOCUS, SHORT_BREAK, and LONG_BREAK types.
Active session	A session where endedAt is null. Only one allowed per user at a time.
Completed session	A session where completed = true (timer ran to 0:00).
Abandoned session	A session where completed = false AND endedAt is set.
Pomodoro	Informal/UI term only. In code, always use 'focus session'. Never use 'pomodoro' as a variable name.
Streak	Consecutive calendar days with at least one completed FOCUS session. Evaluated in user's local timezone.
Daily goal	User-configured daily focus target in minutes (UserSettings.dailyGoalMinutes). Counts FOCUS sessions only, not breaks.
Deep work	Marketing/UI copy term. In code, always equivalent to a completed FOCUS session.
taskName	What the user is working on. Required string on every FocusSession. Shown in session list.
plannedMins	Duration (in minutes) the timer was set to when the session started.
actualMins	Real elapsed duration when the session ended (complete or abandoned). Null if session still active.
weekStartDate	The Monday at 00:00 UTC of the current calendar week. Used as the unique key for AIInsight records.

Agent Instructions & Constraints

Read this section before writing any code. These rules override any other instinct or default behavior.

14.

14.1 Phase Gates

This spec is divided into three phases. Each phase has a gating rule:

- Do NOT start Phase 2 until all Phase 1 requirements pass and test coverage is above 70%.
- Do NOT start Phase 3 until all Phase 2 requirements pass, including the cron job and DailySummary migration.
- When starting a new phase, run the required DB migrations before writing any feature code.

14.2 Build Order Within Each Phase

- Always build in this order: DB schema → API routes → API tests → Frontend → Frontend tests.
- Complete each feature fully (backend + frontend + tests) before moving to the next feature in the same phase.
- After completing each feature, verify it satisfies every requirement ID listed for that feature before moving on.

14.3 Decision Rules

- When the spec is ambiguous, implement the simpler interpretation and add: // SPEC QUESTION: [description of ambiguity]
- When making an architectural decision not covered by this spec, add: // ARCH DECISION: [explanation]
- The spec is the source of truth. If you believe the spec has an error, implement it as written and add: // SPEC ERROR: [concern]
- Do not add libraries not listed in Section 2. If you must, add a // ARCH DECISION: comment and document the dependency.
- Do not implement any feature listed as Out of Scope for the current phase.

14.4 Security Rules (Non-Negotiable)

- Never hardcode secrets, user IDs, or URLs. All config from environment variables.
- All DB queries must scope to the authenticated user's userId. Never query across users.

- The ANTHROPIC_API_KEY must never appear in frontend code, browser requests, or logs.
- Never expose stack traces to users. All production errors return generic messages.

14.5 Handling Gaps

If this spec does not cover a situation you encounter, apply these rules in order:

43. Check if a similar pattern is already established elsewhere in the spec. Follow it.
44. Choose the approach that is most consistent with the existing architecture.
45. Choose the simpler of two reasonable approaches.
46. Leave a comment explaining your choice.

End of Specification —